

CALL ME MAYBE

LEARNING BLUEPRINT

A Systems-First Guide to Building a Constrained-Decoding Function-Calling Engine

Dependency-ordered • Architecture-first • Mental-model driven

Table of Contents

- Before You Start — Research Checklist
- 1. Cover & Orientation
- 2. The Big Picture
- 3. Learning Path Derivation
- Chapter 1 — The Raw Model Interface
- Chapter 2 — Manual Token-by-Token Generation
- Chapter 3 — The Vocabulary and Token-to-String Mapping
- Chapter 4 — JSON Grammar as a State Machine
- Chapter 5 — Constrained Decoding: Masking Invalid Tokens
- Chapter 6 — Schema-Aware Constraints
- Chapter 7 — Function and Parameter Selection by the LLM
- Chapter 8 — The Full Generation Pipeline
- Chapter 9 — Input/Output Contract and CLI
- Chapter 10 — Robustness, Errors, and Packaging
- Final System Integration
- Appendix
- Study & Implementation Timetable

Before You Start — Research Checklist

Assumption: This blueprint assumes no prior knowledge of the topics below. It does not teach them — it tells you they exist and are required, and points you to go learn each one from a source of your choice (documentation, a course, a book, or a tutorial) before starting the matching chapter. Trying to learn a topic below for the first time from inside a chapter will slow you down far more than spending 20-30 minutes on it beforehand.

Each row lists a topic you must be able to explain in your own words before starting the listed chapter, plus one concrete question you should be able to answer once you've researched it — use that question to self-check whether you're ready to move on.

Topic	Needed Before	Self-Check Question
What a tokenizer is, and what a token is (as distinct from a word or character)	Chapter 1	Why might the word "unhappiness" be split into more than one token?
What logits are, and how they differ from probabilities	Chapter 1	What operation turns a list of logits into a list of probabilities that sum to 1?
What autoregressive text generation means (generating one token at a time, feeding it back in)	Chapter 2	If a model has generated 5 tokens so far, how many tokens does it look at to decide the 6th?
What argmax is	Chapter 2	Given the list [0.1, 0.7, 0.2], what index does argmax return?
What a Python dictionary and a Python set are, and roughly how fast lookups are in each	Chapter 3	Which is faster to check membership in for large data: a Python list or a Python set?
What a finite state machine is, in general (outside this project)	Chapter 4	In your own words, what are the two things a finite state machine needs: what it's tracking, and what makes it change?
What JSON grammar looks like structurally (objects, arrays, strings, numbers, booleans, null, and how they nest)	Chapter 4	Is {"a": [1, 2,]} valid JSON? Why or why not?
What negative infinity means as a float value in Python, and why it's used with logits specifically	Chapter 5	If you set a token's logit to negative infinity before an operation that picks the highest logit, can that token ever be chosen?
What a JSON Schema is, at a conceptual level (not full JSON Schema spec — just the idea of describing allowed shapes and types)	Chapter 6	What's the difference between saying a JSON value must be 'a string' versus 'the string "blue"'?
Python type hints and the `typing` module basics (List, Dict, Optional)	Global setup / Chapter 8	What does `Optional[int]` mean as a type hint?
What pydantic is used for, at a conceptual level (data validation via typed models)	Global setup / Chapter 6	If you define a pydantic model with a field typed as `int` and pass it the string "abc", what happens?

Topic	Needed Before	Self-Check Question
Python's argparse module basics (or any CLI-arg-parsing library)	Chapter 9	What's the difference between a positional argument and an optional flag argument (like --input) in a CLI?

If you already know a topic: Skip it. This checklist exists so nothing in the chapters ahead assumes knowledge you haven't been told you need — it is not a mandatory course, it is a readiness check.

1. Cover & Orientation

What Call Me Maybe Is

"Call Me Maybe" is a Python program that turns a natural-language request (e.g. "What is the sum of 2 and 3?") into a structured, machine-executable function call (e.g. {"name": "fn_add_numbers", "parameters": {"a": 2, "b": 3}}). The program is given a list of available functions with their names, parameter types, and descriptions, and a list of natural-language prompts. For every prompt, it must use a small language model (Qwen3-0.6B, 0.6 billion parameters) to choose the correct function and produce correctly-typed arguments, output as valid JSON.

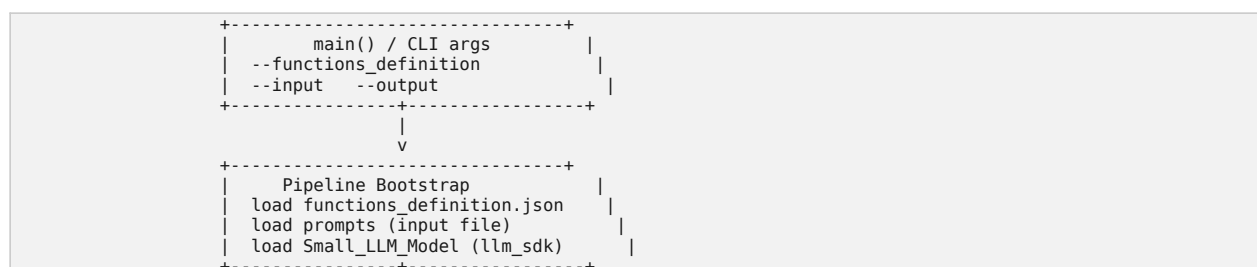
The central technical constraint is that small language models are unreliable at free-form structured output — asked to produce JSON directly, they might succeed only 30% of the time. This project does not solve that with prompting tricks. It solves it with constrained decoding: at every single token the model generates, invalid tokens (ones that would break JSON syntax or violate the function schema) are forcibly excluded before the model is allowed to choose, guaranteeing valid, schema-compliant output on every single generation.

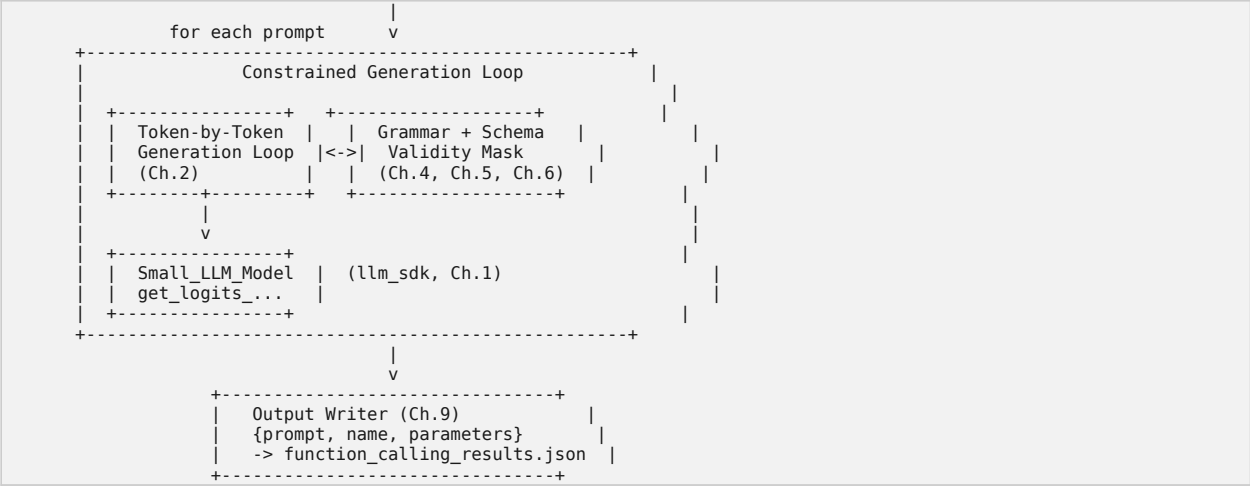
This means the program does not call a high-level `.generate()` convenience function and hope for the best. It builds its own token-by-token generation loop directly on top of raw model logits, and layers a custom validity filter into that loop.

What You Will Be Able to Build

- Access a small LLM's raw next-token logits directly, without any generation helper functions.
- Build your own manual, token-by-token autoregressive generation loop.
- Map between the model's numeric token IDs and their string representations using the vocabulary file.
- Track JSON grammar validity as an explicit state machine, one character/token at a time.
- Mask (set to negative infinity) every logit that would produce an invalid next token, before selecting one.
- Extend that masking to enforce a specific function-calling schema, not just generic JSON validity.
- Have the LLM itself select which function to call and which arguments to extract — never via string matching or heuristics.
- Wire the whole thing into a CLI tool that reads two input files and writes one schema-compliant output file, with robust error handling.

System Architecture Diagram (High Level)





2. The Big Picture

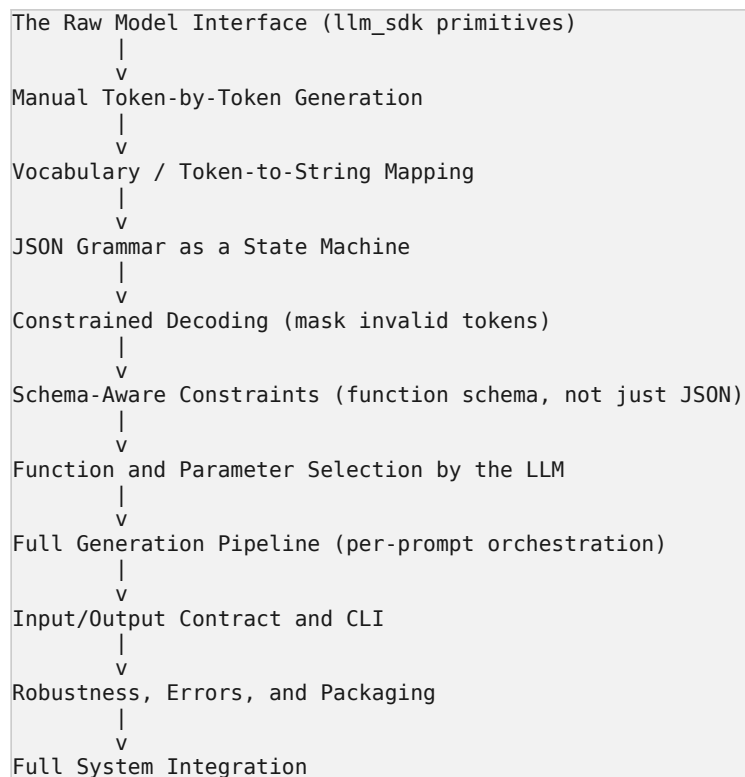
Black-Box Behavior First

From the outside, this program is a command-line tool. Given a JSON file describing available functions and a JSON file of natural-language prompts, it produces one JSON output file. That output file contains, for every input prompt, the exact function name and correctly-typed arguments the LLM decided apply — always valid JSON, always matching the schema, with no exceptions.

Cracking It Open: One Layer Down

Underneath, for every single prompt, the program does not ask the model to "write JSON" and hope. It runs its own generation loop: it repeatedly asks the model for next-token logits, computes — token by token — which of those tokens are still allowed given (a) JSON syntax rules and (b) the function schema, forces every disallowed token's logit down to negative infinity, and only then picks a token. This repeats until a complete, valid function call has been generated.

Full Dependency Graph



Why This Order

Each layer above is a strict prerequisite for the layer below it:

- You cannot build a generation loop before you understand what the SDK actually gives you (raw logits only — no generate() helper) — the SDK's shape determines the loop's shape.

- You cannot mask invalid tokens before you can translate a token ID into the string it represents, because masking decisions are made on strings/characters, but application happens on numeric token IDs.
- You cannot enforce JSON validity before you have a precise model of what 'valid so far' means at every partial state — that's the state machine.
- You cannot enforce schema validity (correct function names, correct parameter keys, correct types) until generic JSON masking already works, because schema constraints are additional restrictions layered on top of, not instead of, JSON grammar validity.
- You cannot wire the full per-prompt pipeline together until a single call's generation is fully correct in isolation — testing the loop on one prompt must happen before batching many prompts.

Ordering rule: A module is placed after every module whose behavior it depends on, and before every module that depends on it. File order and subject order are not used.

3. Learning Path Derivation

Day-independent module order (see Timetable section for day mapping):

- [1] The Raw Model Interface
- [2] Manual Token-by-Token Generation
- [3] Vocabulary and Token-to-String Mapping
- [4] JSON Grammar as a State Machine
- [5] Constrained Decoding: Masking Invalid Tokens
- [6] Schema-Aware Constraints
- [7] Function and Parameter Selection by the LLM
- [8] The Full Generation Pipeline
- [9] Input/Output Contract and CLI
- [10] Robustness, Errors, and Packaging
- [11] Full System Integration

Each chapter below follows a fixed structure: You Are Here, Why This Module Exists, Conceptual Understanding, Conceptual Checkpoint, Implementation Goals, System Reconnection, Testing & Verification, Common Mistakes & Debugging, Done Criteria, What This Unlocks.

Chapter 1 — The Raw Model Interface

You Are Here

Nothing has been built yet. This is the foundation layer. Before any generation loop, any masking, or any schema logic can exist, you need to understand exactly what the provided `llm_sdk` gives you access to — and just as importantly, what it deliberately does not give you. This chapter unlocks every subsequent chapter, since all of them call into this SDK.

Why This Module Exists

The subject is explicit that your solution must not rely on the model spontaneously producing correct JSON from a prompt, and must not use any generation helper beyond what `llm_sdk` exposes. The SDK gives you only low-level primitives: `encode`, `decode`, `get_logits_from_input_ids`, and vocabulary file paths. There is no `.generate()` method, no built-in sampling, no built-in stopping logic. Understanding this precisely, before writing any code, determines the shape of everything else you build.

Conceptual Understanding

What Small_LLM_Model Actually Exposes

The SDK provides a wrapper class around a Hugging Face causal language model. Its public methods, exactly as documented in the subject:

`encode(text: str) -> Tensor` — tokenizes a string into a tensor of input token IDs.

`decode(token_ids: List[int]) -> str (optional)` — the inverse: turns token IDs back into text.

`get_logits_from_input_ids(input_ids: List[int]) -> List[float]` — given the full sequence of token IDs generated so far, returns the raw logits for what the next token should be. This is the only way you get information out of the model about what comes next.

`get_path_to_vocab_file()` -> str — returns a filesystem path to a file mapping token IDs to their string forms. This is required, not optional, for constrained decoding, and is covered in Chapter 3.

What Is Deliberately Absent

There is no method that takes a prompt and returns a finished response. There is no sampling strategy built in (no temperature, no top-k, no built-in argmax). There is no automatic stopping condition. Every one of these must be built by you, on top of the raw logits the SDK exposes. This absence is not a gap to work around — it is the entire point of the project: you are the one responsible for turning raw, unreliable logits into guaranteed-valid structured output.

Logits Are Not Probabilities

`get_logits_from_input_ids` returns raw, unbounded scores — one float per token in the model's vocabulary — not probabilities. A higher logit means the model considers that token more likely to come next, but the values themselves are not between 0 and 1 and do not sum to anything meaningful on their own. For basic generation, you do not need to convert logits to probabilities at all: selecting the token with the single highest logit (argmax, covered in Chapter 2) works directly on raw logits, since the ordering of logits and the ordering of their corresponding probabilities is always the same.

The Forbidden Boundary

The subject forbids private methods or attributes from `llm_sdk` — meaning anything on the class not listed in its public interface. It also forbids importing `torch`, `transformers`, `huggingface` libraries, `dspy`, `outlines`, or similar packages directly into your own code. All model access must go through the four public methods above. This is a hard constraint on every later chapter: whatever you build, you build it using only these four operations.

Conceptual Checkpoint

1. Name all four public methods of `Small_LLM_Model` and, for each, state exactly what it takes as input and returns as output.
2. Why does the SDK not provide a `.generate()` method, given that the subject explicitly wants you to build your own generation loop?
3. Why can you select the highest-logit token directly, without first converting logits to probabilities, if all you need is the single most likely next token?
4. What is forbidden in terms of what packages your own code may import directly?

Implementation Goals

- ☐ A small script or module that instantiates `Small_LLM_Model` once.
- ☐ A call to `encode()` on a sample prompt, confirming it returns a token ID sequence.
- ☐ A call to `get_logits_from_input_ids()` on that sequence, confirming it returns a list of floats matching the model's vocabulary size.
- ☐ A quick manual inspection (e.g. printing the indices of the 5 highest logits) to build intuition for what the raw output looks like before writing any generation logic.

System Reconnection — Where does this fit into the whole project?

This chapter builds the innermost box in the architecture diagram — `Small_LLM_Model` — that everything else calls into. Chapter 2's generation loop calls `get_logits_from_input_ids()` every iteration. Chapter 3's vocabulary mapping uses `get_path_to_vocab_file()`. No later chapter bypasses this interface; they all build on top of exactly these four methods.

Testing & Verification

- Encode a simple, known prompt and print the resulting token IDs — confirm the count is reasonable for the prompt's length.
- Call `get_logits_from_input_ids()` and confirm the returned list length matches the model's vocabulary size (you can cross-check this against the vocabulary file from Chapter 3 once you reach it).

Common Mistakes & Debugging Strategies

- Assuming logits are probabilities and trying to interpret their raw values as percentages — they are not bounded to `[0, 1]`.
- Looking for a `generate()` or similar convenience method on the SDK and not finding one — this is expected, not a bug in the SDK.
- Accidentally importing `transformers` or `torch` directly in your own code to "help" — this violates the subject's explicit restriction, even if it would technically work.

Done Criteria

- ☐ You can encode a prompt, retrieve logits for the next token, and explain in your own words what each number in the returned list represents.
- ☐ You can state precisely, without checking the SDK again, which four methods are available and what each does.

What This Unlocks

Chapter 2 (Manual Token-by-Token Generation), which turns single logit calls into a repeating loop that produces full sequences.

Chapter 2 — Manual Token-by-Token Generation

You Are Here

You understand what the SDK gives you: encode, decode, and raw next-token logits, one call at a time. This chapter turns that single call into a loop that produces a full sequence of tokens, without yet worrying about JSON validity or schema — that comes later. This is unconstrained generation, built entirely by you.

Why This Module Exists

Language models generate text autoregressively: one token at a time, where each new token depends on every token generated before it, including tokens the model itself generated earlier in the same sequence. The subject requires you to build this loop yourself, since no `generate()` helper exists. Getting this loop correct and well-understood, without constraints, is a prerequisite for adding constraints on top of it in Chapters 4-6.

Conceptual Understanding

The Autoregressive Loop

At each step, the current full sequence of token IDs (the original prompt plus every token generated so far) is passed to `get_logits_from_input_ids()`. This returns logits for what the very next token should be. One token is selected from those logits (Chapter 2 uses simple `argmax`; later chapters replace this selection step with a constrained version). The selected token is appended to the sequence, and the loop repeats using the new, longer sequence.

```
tokens = encode(prompt)           # initial sequence
while not done:
    logits = get_logits_from_input_ids(tokens)
    next_token = argmax(logits)    # simple selection, unconstrained
    tokens.append(next_token)
    if next_token == EOS_TOKEN_ID or len(tokens) >= max_length:
        done = True
```

Why Every Generated Token Must Be Fed Back In

The model has no memory between calls to `get_logits_from_input_ids()` — it is a pure function of the input sequence you give it. If you generated token 6 but didn't include it in the input sequence for computing token 7's logits, the model would be predicting token 7 as if token 6 never existed, producing incoherent output. The full growing sequence must always be passed in full on every call.

Argmax as the Simplest Selection Strategy

Argmax selects the single token with the highest logit value at each step. This is deterministic (the same input always produces the same output) and is sufficient for this project, since you don't need creative or varied text — you need one single, precise, structurally correct answer per prompt. More sophisticated sampling strategies (temperature, top-k, top-p) exist but are unnecessary complexity here, and would only make guaranteeing valid structure harder, not easier.

Stopping Conditions

A generation loop needs to know when to stop. Two conditions are typical: reaching an end-of-sequence (EOS) token the model itself generates, or reaching a maximum token length safety limit to avoid infinite loops. In this project, once constrained decoding is added (Chapter 5), there is a third, more precise

stopping condition specific to this project: the JSON grammar state machine reaching a complete, closed top-level structure. That stopping condition replaces reliance on the model's own EOS token, since a small model cannot be trusted to emit EOS at exactly the right structural moment.

Conceptual Checkpoint

1. Describe the autoregressive loop in your own words: what goes in, what comes out, and what happens to the output before the next iteration.
2. Why must every previously generated token be included in the input to the next logits call?
3. What does `argmax` select, precisely, from a list of logits?
4. Name two different reasons a generation loop might need to stop, and state which one this project will eventually replace with something more precise.

Implementation Goals

- ☐ A generation function taking a prompt string and a `max_length`, returning a full generated token sequence using unconstrained `argmax` selection.
- ☐ A basic stopping condition (max length, and/or EOS token if the tokenizer exposes one via `decode()`).
- ☐ Decoding the final token sequence back to a string using `decode()`, to visually inspect what unconstrained generation produces.

System Reconnection — Where does this fit into the whole project?

This chapter builds the core loop shown in the architecture diagram's "Token-by-Token Generation Loop" box. Every later chapter that adds constraints (Chapters 4-6) modifies only the token-selection step inside this loop — the loop's overall shape (get logits, select, append, repeat) does not change.

Testing & Verification

- Run the loop on a simple open-ended prompt (not related to JSON yet) and confirm the decoded output is coherent, readable text of roughly the expected length.
- Run the loop twice on the same prompt and confirm you get identical output both times — `argmax` is deterministic, so this should always hold.

Common Mistakes & Debugging Strategies

- Passing only the newly generated token to `get_logits_from_input_ids()` instead of the full growing sequence — this breaks the model's ability to use prior context.
- Forgetting a maximum length safety limit, risking an infinite loop if the model never naturally produces a stopping signal.
- Confusing `decode()` (turns token IDs into a readable string, useful for debugging) with the token IDs themselves (what the loop actually operates on) — don't decode mid-loop and try to re-encode; work with token IDs throughout the loop.

Done Criteria

- ☐ The generation loop runs to completion on a test prompt without crashing or hanging.
- ☐ Decoded output is coherent, unconstrained natural language — you are not yet expecting valid JSON at this stage.

☐ Running the same prompt twice produces identical output, confirming deterministic argmax selection.

What This Unlocks

Chapter 3 (Vocabulary and Token-to-String Mapping), which is needed before token selection can be made JSON-aware, since JSON validity decisions are made on the string a token represents, not on its numeric ID directly.

Chapter 3 — The Vocabulary and Token-to-String Mapping

You Are Here

You have a working, unconstrained generation loop. Before you can restrict that loop to only ever produce valid JSON, you need a fast way to answer the question: "what string does token ID 4771 actually represent?" This chapter builds that mapping, in isolation, before it's wired into any masking logic.

Why This Module Exists

Constrained decoding decisions ("is this token allowed right now, given JSON grammar rules?") are naturally made by looking at what string a token represents — a token representing ```` behaves very differently from a token representing ``7``. But the model only understands and produces numeric token IDs. You need a reliable, fast mapping in both directions to bridge this gap, and it must be efficient, since it will be consulted at every single token generated, for every single prompt.

Conceptual Understanding

What `get_path_to_vocab_file()` Gives You

This SDK method returns a filesystem path to a file (typically a JSON file for this class of tokenizer) that lists every token in the model's vocabulary and its corresponding string form. This file is what makes offline, string-level reasoning about tokens possible without calling `decode()` repeatedly during generation, which would be slow inside a tight per-token loop.

Building a Fast Token ID to String Map

Load the vocabulary file once, at startup, and build an in-memory mapping (a Python dictionary) from token ID to its string form. This must be built once and reused for every prompt and every generation step — rebuilding it per-token or per-prompt would be wasteful and would slow generation down significantly, working against the subject's reasonable-speed requirement.

Why This Must Be a Dictionary, Not a Search

A generation loop may run this lookup hundreds of times per prompt, across potentially many prompts. A linear search through a list of (id, string) pairs for every lookup would be far slower than a dictionary lookup, which is effectively constant-time regardless of vocabulary size. This directly affects whether your solution can process all test prompts in a reasonable time, which the subject explicitly requires.

Special Tokens and Encoding Quirks

Some tokens don't represent ordinary readable text — special tokens like padding, end-of-sequence, or beginning-of-sequence markers exist in most vocabularies. Additionally, as the subject notes, tokenizers frequently encode a leading space as part of a token itself (for example, a token might represent `" hello"` including the space, distinct from a token representing `"hello"` without one). Your masking logic in later chapters must account for the actual string content of a token, including any leading-space marker character the tokenizer uses, rather than assuming tokens map cleanly to whole words or single characters.

Conceptual Checkpoint

1. What does `get_path_to_vocab_file()` return, and what do you do with what it returns?

2. Why is a dictionary lookup preferred over a linear search for token ID to string mapping in this project specifically?
3. Why can't you assume every token corresponds to exactly one visible character or one whole word?
4. What kind of tokens exist in a vocabulary beyond ordinary readable text?

Implementation Goals

- ☐ A function that loads the vocabulary file (via `get_path_to_vocab_file()`) once and builds a token ID -> string dictionary.
- ☐ A reverse mapping (string -> token ID or token IDs, since multiple IDs could theoretically map to related strings) if needed for your masking approach.
- ☐ A helper function answering, for a given token ID, "what string does this represent," using the dictionary — not repeated calls to `decode()`.
- ☐ Confirmation that this vocabulary loading happens once per program run, not once per prompt or once per token.

System Reconnection — Where does this fit into the whole project?

This chapter fills in a supporting utility used by every masking decision from Chapter 5 onward. It does not sit as its own box in the high-level architecture diagram — it is the lookup table the Grammar + Schema Validity Mask box relies on internally. Every time that box asks "would selecting this token be valid," it is consulting the mapping built here.

Testing & Verification

- Pick a handful of token IDs, look up their string forms using your dictionary, and manually cross-check a few against `decode()` on a single-token list to confirm they match.
- Time how long building the full dictionary takes at startup versus how long a single dictionary lookup takes afterward — confirm the lookup itself is effectively instant.

Common Mistakes & Debugging Strategies

- Calling `decode()` inside the per-token masking loop instead of using a prebuilt dictionary — this will be correct but far too slow across many tokens and many prompts.
- Assuming a token's string form never includes a leading space or punctuation, and being surprised later when JSON-character-level masking behaves unexpectedly.
- Rebuilding the vocabulary dictionary inside the per-prompt loop instead of once at program startup.

Done Criteria

- ☐ A single, reusable token ID to string dictionary exists, built once per program run.
- ☐ Given any token ID from the model's vocabulary, you can retrieve its string form in effectively constant time.
- ☐ You can explain, with an example, why a token's string form might include a leading space or partial word.

What This Unlocks

Chapter 4 (JSON Grammar as a State Machine), which uses this string mapping to decide, character by character, whether a candidate token would keep the generated output on a path toward valid JSON.

Chapter 4 — JSON Grammar as a State Machine

You Are Here

You can generate tokens in a loop, and you can translate any token ID into the string it represents. This chapter builds the logic that tracks, at every point during generation, exactly what JSON syntax allows to come next — independent of any function schema, which comes later in Chapter 6. This is the JSON-validity layer, built and tested in isolation before it's wired into token masking in Chapter 5.

Why This Module Exists

The subject requires 100% valid JSON output, not "usually valid." A small model cannot be trusted to produce this on its own. To guarantee it, you need a precise, mechanical model of what "valid so far" means at every partial point during generation — not just at the very end. This is what a state machine gives you: a way to know, after every single character, exactly which characters are legal next.

Conceptual Understanding

Why 'Valid at the End' Is Not Enough

It's not sufficient to check whether the final output is valid JSON after generation completes — by then it's too late to fix a wrong token that was already generated and fed back into the model. Constrained decoding requires knowing, before each token is chosen, whether choosing it would still leave a path open to eventually complete valid JSON. This is why the state machine must track validity incrementally, one character at a time, as generation proceeds — not just validate a finished string.

Core JSON States

A simplified JSON grammar state machine needs states such as: expecting a value (could be an object, array, string, number, boolean, or null); inside an object, expecting a key or a closing brace; expecting a colon after a key; expecting a value after a colon; inside a string (with a substate for "just saw a backslash, expect an escape character next"); inside a number (with substates for sign, digits, decimal point, exponent); after a complete value, expecting a comma or a closing brace/bracket; inside an array, expecting a value or a closing bracket. Each state defines a specific, limited set of legal next characters.

Transitions

A transition is a rule: given the current state and the next character, what is the new state? For example, if the state is "inside an object, expecting a key," the only legal next character is a double-quote (starting a string key) or, if the object could be empty, a closing brace. Seeing a double-quote transitions to "inside a string, as a key." Seeing anything else in that state is invalid and must never be allowed.

```
state = EXPECT_VALUE
for char in candidate_output:
    if char not in legal_next_chars(state):
        # this path is invalid - this character can never be chosen here
        reject()
    state = transition(state, char)
```

Operating at the Character Level, Applied at the Token Level

JSON grammar rules are naturally defined character by character, but the model generates tokens, which may represent multiple characters at once (or even partial characters in some tokenizer schemes). This means: for each candidate token, you must check whether every character in that token's string form would be individually valid given the current state, advancing the state machine

through each character in turn — not just check the token as one opaque unit. A token is only a legal choice if its entire string content keeps the state machine in a valid path from start to finish.

Conceptual Checkpoint

1. Why is it not sufficient to check JSON validity only after the full output is generated?
2. List at least five distinct states a JSON grammar state machine needs to track partial validity correctly.
3. Given the state "inside an object, expecting a key," what are the legal next characters, and what state does seeing a double-quote transition to?
4. Why must a state machine advance character by character through a token's string content, rather than treating each token as one indivisible unit?

Implementation Goals

- ☐ A JSON state enum or equivalent representing all the distinct grammar states needed (start, expect-key, expect-colon, expect-value, in-string, in-string-escape, in-number, after-value, in-array, done, etc.).
- ☐ A transition function taking (current_state, character) and returning (new_state) or an explicit "invalid" signal.
- ☐ A function that, given a state and a full candidate string (e.g. a token's string form), determines whether processing that entire string keeps the state machine on a valid path, and if so, what the resulting state is.
- ☐ No masking or model interaction yet — this chapter is purely about correctly modeling JSON grammar as data and transition rules.

System Reconnection — Where does this fit into the whole project?

This chapter builds the JSON-grammar half of the Grammar + Schema Validity Mask box in the architecture diagram. Chapter 5 will consult this state machine, alongside the vocabulary mapping from Chapter 3, to decide which tokens are legal at each generation step. Chapter 6 will add a second, schema-aware layer on top of this one, without needing to modify this chapter's logic.

Testing & Verification

- Manually feed the state machine a handful of known-valid JSON strings, character by character, and confirm it never rejects a character and ends in a valid "done" state.
- Manually feed it a handful of deliberately broken JSON strings (trailing commas, unclosed braces, missing colons) and confirm it correctly rejects at the exact point the error occurs.

Common Mistakes & Debugging Strategies

- Forgetting the escape-character substate inside strings, causing a backslash followed by a quote to be misread as the string ending.
- Treating numbers too simply and not accounting for valid but easy-to-miss forms (negative numbers, decimals, exponents) versus genuinely invalid ones (leading zeros in some strict JSON interpretations, trailing decimal points).
- Checking token validity as an atomic unit instead of char-by-char, causing a multi-character token that starts validly but ends invalidly to be incorrectly accepted.

Done Criteria

- ☐ The state machine correctly accepts a range of hand-written valid JSON strings, character by character, with no false rejections.
- ☐ The state machine correctly rejects a range of hand-written invalid JSON strings, exactly at the character where they become invalid.
- ☐ You can trace, for any partial JSON string, exactly what state the machine is in and what characters would be legal next.

What This Unlocks

Chapter 5 (Constrained Decoding: Masking Invalid Tokens), which uses this state machine's per-character validity checks to determine, for every candidate token during generation, whether selecting it is allowed.

Chapter 5 — Constrained Decoding: Masking Invalid Tokens

You Are Here

You have an unconstrained generation loop, a fast token-to-string lookup, and a JSON grammar state machine that can judge any candidate string's validity. This chapter connects all three: at every step of generation, invalid tokens are excluded before the model is allowed to choose, guaranteeing syntactically valid JSON output. This is the first point where the full constrained-decoding mechanism actually runs.

Why This Module Exists

The subject's central requirement is 100% valid JSON, achieved specifically through constrained decoding — modifying the model's logits before selection, not through prompting or post-hoc correction. This chapter is where that mechanism is actually implemented and first becomes observable: generation that was previously free-form now becomes forcibly, structurally valid.

Conceptual Understanding

The Masking Step, Precisely

At each generation step: get the current logits (Chapter 1/2). For every token ID in the vocabulary, look up its string form (Chapter 3) and check whether appending it to the current output, given the current JSON state, would remain valid (Chapter 4). If not valid, set that token's logit to negative infinity. Once every invalid token has been masked, apply the same selection step as before (argmax, Chapter 2) — but now, argmax can only ever land on a token that was left unmasked, i.e. a token guaranteed to keep the output on a valid path.

```
logits = get_logits_from_input_ids(tokens)
for token_id in range(len(logits)):
    token_str = vocab_map[token_id]
    if not json_state.would_remain_valid(token_str):
        logits[token_id] = float('-inf')
next_token = argmax(logits)  # can only select a valid token now
```

Why Negative Infinity Specifically

Setting a logit to negative infinity guarantees that token can never be selected by argmax (or any probability-based sampling, since it corresponds to a probability of exactly zero after any standard normalization), regardless of how large or small the other logits are. A very large negative number would usually work in practice, but negative infinity removes any possibility of numerical edge cases where an unmasked token's legitimately very low logit could still accidentally exceed an insufficiently negative mask value.

This Must Update the State Machine Too

After a token is selected, its string content must be run through the JSON state machine's transition logic (Chapter 4) to update the current state, exactly as if you were validating it — because you are. The state after selecting a token becomes the starting state for masking the next token. Skipping this update means every subsequent masking decision uses stale, incorrect state.

Performance: Masking the Full Vocabulary Every Step Is Expensive

A vocabulary can contain tens of thousands of tokens, and this masking check runs at every single generation step, for every prompt. Checking every token's full string against the state machine on every

step is the most expensive part of this pipeline. Where possible, narrow the set of tokens you need to check per state (for example, once inside a numeric value, only tokens whose string content is entirely digits, a decimal point, minus sign, or a valid closing character need to be checked at all — most of the vocabulary can be trivially excluded without a full character-by-character state machine run). This matters directly for meeting the subject's reasonable-speed requirement.

Conceptual Checkpoint

1. Walk through the masking step in order: what happens first, what happens to invalid tokens, and what happens last.
2. Why is negative infinity used instead of just a very large negative number?
3. What must happen to the JSON state machine immediately after a token is selected, and why does skipping this step corrupt all future masking decisions?
4. Why is masking the full vocabulary on every single step potentially a performance problem, and what's one way to reduce that cost?

Implementation Goals

- ☐ A masking function combining the vocabulary map (Ch.3), the JSON state machine (Ch.4), and raw logits (Ch.1) to produce a fully masked logits array.
- ☐ Integration of this masking function into the generation loop from Chapter 2, replacing plain unconstrained argmax with masked argmax.
- ☐ State machine advancement wired in immediately after each token selection, keeping JSON state accurate across the whole loop.
- ☐ At least one performance optimization narrowing which tokens need full validation per state, rather than checking the entire vocabulary from scratch every step.

System Reconnection — Where does this fit into the whole project?

This chapter completes the core of the Grammar + Schema Validity Mask box shown in the architecture diagram, for its generic-JSON half. Combined with Chapter 2's loop, this produces the first version of the system that can generate syntactically valid JSON, on every run, guaranteed — though not yet schema-aware, which is Chapter 6's job.

Testing & Verification

- Run the fully constrained loop (Chapters 2, 4, 5 together) on a prompt asking the model to produce some JSON, with no schema constraint yet. Confirm the output is syntactically valid JSON, parseable without errors, across at least 20 repeated runs.
- Deliberately inspect a few masking steps mid-generation (e.g. print how many tokens were masked out at a given step) to build confidence the masking logic is actually doing meaningful work, not just passing everything through.

Common Mistakes & Debugging Strategies

- Forgetting to update the JSON state machine after token selection, causing all subsequent masking to operate on stale state and eventually produce invalid output despite masking being "in place."

- Off-by-one errors between checking validity of "token appended" versus "token about to be appended" — be precise about whether you're validating before or after the hypothetical append.
- Masking every single generation step against the entire vocabulary with no optimization, resulting in generation that is technically correct but too slow to meet the subject's performance requirement across many prompts.

Done Criteria

- ☐ Across at least 20 repeated generations on prompts requesting JSON output, 100% of outputs are syntactically valid, parseable JSON.
- ☐ You can point to the exact moment in your code where an invalid token is excluded and explain why it was excluded.
- ☐ Generation speed across a batch of prompts remains reasonable (the subject's benchmark: all test prompts processed in under 5 minutes on standard hardware).

What This Unlocks

Chapter 6 (Schema-Aware Constraints), which layers function-schema-specific restrictions on top of this now-working generic JSON validity mask.

Chapter 6 — Schema-Aware Constraints

You Are Here

Generic JSON validity is guaranteed by Chapter 5's masking. This chapter narrows that further: not just any valid JSON, but JSON that matches the exact function-calling schema loaded from `functions_definition.json` — correct top-level keys, correct function names, correct parameter names, correct value types per parameter. This is layered on top of, not instead of, the JSON grammar layer.

Why This Module Exists

Valid JSON alone is not enough. The subject requires the output to have exactly the keys `prompt`, `name`, and `parameters`, where `name` must be one of the real available function names and `parameters` must contain exactly the arguments that function requires, with correct types. A syntactically valid but semantically wrong JSON object (e.g. a made-up function name, or a string where a number was required) still fails the subject's validation rules.

Conceptual Understanding

The Schema, as Loaded From `functions_definition.json`

Each function definition provides: a name (e.g. `fn_add_numbers`), a description, a `parameters` object listing each parameter's name and type (e.g. `{"a": {"type": "number"}, "b": {"type": "number"}}`), and a `returns` type. This schema is loaded once, validated (ideally with `pydantic`, per the subject's requirement that all classes use it), and becomes the second constraint layer, alongside the JSON grammar state machine from Chapter 4.

Restricting the Output's Top-Level Shape

The output JSON for each prompt must have exactly three keys: `prompt`, `name`, and `parameters` — no more, no fewer. Since `prompt` is known ahead of time (it's the original input, not something the model needs to generate), the model-driven generation typically only needs to produce the `name` and `parameters` portion; the `prompt` field can be filled in directly from the input rather than generated by the model at all. This significantly narrows what the model actually needs to generate correctly.

Restricting Function Names to Known Values

Once the state machine reaches the point of generating the value for the `"name"` key, only strings that are prefixes of an actual known function name (and eventually match one exactly) should be allowed. This means the token-level masking check, at this specific point in generation, must consult not just generic JSON string rules but the literal, finite list of valid function names — rejecting any token that would make the generated string deviate from every real function name.

```
# At the 'inside the name value string' state:
candidate = current_partial_name_string + token_str
valid_function_names = [f.name for f in schema.functions]
is_allowed = any(name.startswith(candidate) for name in valid_function_names)
```

Restricting Parameter Keys and Value Types

Once the model has committed to a specific function name, the set of legal keys inside `"parameters"` becomes exactly that function's declared parameter names — no others are valid. For each parameter key, once its value is being generated, the value's type must match the schema (a `"number"` parameter must produce digits/decimal characters forming a valid JSON number, not a quoted string; a `"string"`

parameter must produce a quoted JSON string). This means the schema constraint depends on which function was already chosen, and which key within "parameters" is currently being generated — the set of legal next tokens changes dynamically as generation proceeds through the object.

Composing Two Constraint Layers

At every generation step, a token is only allowed if it passes both the JSON grammar check (Chapter 4/5) and the schema check (this chapter). These are separate, composable checks — the JSON layer doesn't know anything about function names or parameter types, and the schema layer assumes the JSON layer has already ruled out anything syntactically broken. Keeping them separate, as separate chapters and separate code, is what made Chapter 5 independently testable before this chapter's added complexity arrived.

Conceptual Checkpoint

1. What are the exact three top-level keys required in each output object, and which one can be filled in directly without model generation?
2. How does the set of legal function-name tokens get narrowed as the model generates the "name" value, character by character?
3. Once a function name has been chosen, what determines which parameter keys are legal, and what determines which token types are legal for each parameter's value?
4. Why are the JSON grammar layer (Chapter 4/5) and the schema layer (this chapter) kept as separate, composable checks rather than merged into one?

Implementation Goals

- ☐ A pydantic model (or models) representing the loaded function schema: function name, parameters (name + type per parameter), and return type.
- ☐ A schema-tracking state, parallel to the JSON grammar state, tracking which function (if any) has been committed to, and which parameter key (if any) is currently being filled.
- ☐ A schema-aware masking check that narrows legal tokens for function-name generation to only those consistent with real function names.
- ☐ A schema-aware masking check that narrows legal parameter keys to only that function's actual declared parameters, and narrows each parameter's value tokens to match its declared type.
- ☐ Composition of this schema check with the Chapter 5 JSON grammar check, so a token must pass both to be considered valid.

System Reconnection — Where does this fit into the whole project?

This chapter completes the Grammar + Schema Validity Mask box in the architecture diagram, adding its schema-specific half on top of Chapter 4/5's generic JSON half. Together, these two layers guarantee that every generated output is both syntactically valid JSON and semantically valid according to the real function schema — satisfying the subject's requirement that keys and types must match the schema exactly.

Testing & Verification

- Run constrained generation against the real `functions_definition.json` for a prompt clearly matching one function (e.g. "What is the sum of 2 and 3?") and confirm the generated name exactly matches `fn_add_numbers` and parameters contains exactly `a` and `b` as numbers.
- Repeat across all five example functions in `functions_definition.json` (`fn_add_numbers`, `fn_greet`, `fn_reverse_string`, `fn_get_square_root`, `fn_substitute_string_with_regex`) and confirm each produces schema-correct output for a matching prompt.
- Deliberately try a prompt that's ambiguous between two functions and inspect what the model chooses — this doesn't need to be "correct" in a strict sense yet, but the chosen function's schema must still be fully respected in the output.

Common Mistakes & Debugging Strategies

- Allowing the function-name string to be generated freely and only validating it after the fact — this is post-hoc checking, not constrained decoding, and does not guarantee correctness the way the subject requires.
- Forgetting that parameter key order or type coercion (e.g. generating "40" as a string when the schema says number) still constitutes a schema violation even though it's syntactically valid JSON.
- Hardcoding the five example functions instead of reading dynamically from the loaded `functions_definition.json` — the subject explicitly warns that input files may change during peer review.

Done Criteria

- ☐ For every function in a given `functions_definition.json`, at least one matching prompt produces output with the exact correct function name and exactly the correct parameter keys with correctly-typed values.
- ☐ No generated output ever contains a function name that doesn't exist in the loaded schema, or a parameter key that function doesn't declare.
- ☐ The schema logic reads entirely from the loaded `functions_definition.json` at runtime — no function names or parameter names are hardcoded in your masking logic.

What This Unlocks

Chapter 7 (Function and Parameter Selection by the LLM), which addresses the remaining question this chapter doesn't fully answer: ensuring the function chosen is actually the semantically correct one for the prompt, not merely a schema-valid one.

Chapter 7 — Function and Parameter Selection by the LLM

You Are Here

Every generated output is now guaranteed syntactically valid JSON and schema-compliant — but schema compliance alone doesn't guarantee the model picked the right function or extracted the right argument values from the prompt. This chapter is about correctness of choice, not validity of structure, and about making sure that choice is genuinely the model's, not a heuristic in disguise.

Why This Module Exists

The subject states explicitly: "The function to call should be chosen using the LLM, not with heuristics or any other sort of medieval magic." Constrained decoding (Chapters 4-6) guarantees the shape of the output is always valid — it does not, by itself, guarantee the content is correct. A constrained generation process could still validly select the wrong function or extract the wrong numbers, and still pass every JSON/schema check. This chapter is about the prompt design and generation setup that gives the model the best chance of making the correct choice, while keeping that choice entirely within the model's own token generation.

Conceptual Understanding

What 'Heuristics' Would Look Like, and Why They're Forbidden

A heuristic approach might scan the prompt for the word "sum" or "add" and directly select `fn_add_numbers` via a string match, only using the model to extract the numeric arguments, or not using the model at all for function selection. This is explicitly forbidden. The function name itself must be a token sequence the model generated under the schema constraint from Chapter 6 — the constraint narrows what's allowed, but which allowed option gets chosen must come from the model's own logits at each step, not from a separate string-matching function you wrote.

How the Model 'Chooses' Under Constrained Decoding

At each generation step where multiple function names remain possible (e.g. after generating just `fn_`, several function names could still match), the model's own logits — masked to only the currently-valid options — determine which token is selected via `argmax`. If the model's logits genuinely favor tokens consistent with `fn_add_numbers` over `fn_greet` at that point, that preference is what drives the choice. The correctness of that choice depends on how well the model understands the prompt in context, which depends heavily on what other information is present in the generation prompt itself — the actual text fed to `encode()` before generation begins.

Prompt Construction Matters Here

The text you construct and pass to `encode()` before generation starts should include the available function definitions (names, parameter names/types, descriptions) so the model has the necessary information in context to make an informed choice, and the user's natural-language request. How clearly and completely this information is presented directly affects how often the model's unconstrained preferences (which the mask then filters, but does not invent) actually align with the correct function. This is prompt engineering in service of a still fully model-driven, still fully constrained decision — not a way to sneak heuristics back in.

Argument Extraction Is the Same Kind of Choice

Once a function is chosen and its parameter schema is known (Chapter 6), extracting the correct values (e.g. the number 40 from "What is the sum of 40 and 2") is the same kind of model-driven, constrained choice: the schema restricts the value to a valid number token sequence, and the model's own logits, given the full prompt in context, determine which specific digits get generated. No separate regex or string-parsing step should be substituted for this — the value must emerge from the same constrained generation process as the function name did.

Conceptual Checkpoint

1. In your own words, what would a 'heuristic' approach to function selection look like, and why does the subject forbid it?
2. At the point where multiple function names are still schema-valid, what actually determines which one gets selected?
3. Why does the content and clarity of the text passed to `encode()` affect function-selection accuracy, even though selection is still fully constrained and fully model-driven?
4. Why should argument value extraction go through the same constrained generation process as function name selection, rather than a separate parsing step?

Implementation Goals

- ☐ A prompt-construction function that combines the loaded function definitions (names, parameters, descriptions) and the user's natural-language request into the text passed to `encode()`.
- ☐ Confirmation that function-name selection, when multiple functions remain schema-valid at a given point, is driven entirely by the model's own masked logits — no separate string-matching or keyword-detection code path exists anywhere in your function-selection logic.
- ☐ Confirmation that argument values are extracted via the same constrained generation loop, not via regex or manual string parsing of the original prompt.

System Reconnection — Where does this fit into the whole project?

This chapter doesn't add a new box to the architecture diagram — it refines how the existing Constrained Generation Loop is fed (via prompt construction) and confirms that no code path anywhere bypasses the model's own token choices. It directly supports the subject's near-perfect accuracy requirement (90%+ correct function selection and argument extraction), which depends on decision quality, not just structural validity.

Testing & Verification

- Run all 11 example prompts from `function_calling_tests.json` through the full pipeline and manually verify each one selects the correct function and extracts the correct argument values.
- Deliberately construct a new prompt not in the example set, targeting a function in your schema, and confirm correct selection — this tests generalization, not just memorization of the given examples.
- Review your own code for any string-matching, keyword-detection, or regex-based shortcut that could be influencing function selection, and remove it if found — even if it happens to improve accuracy, it violates the subject's requirement.

Common Mistakes & Debugging Strategies

- Adding a "fallback" heuristic for cases where the model seems uncertain, reasoning that it's just a safety net — this is still a forbidden heuristic and will fail the subject's requirement regardless of intent.
- Constructing a prompt for the model that's too sparse (e.g. omitting parameter descriptions), leading to correct JSON structure but frequently wrong function or argument choices.
- Extracting argument values via a regex pulled from the original natural-language prompt instead of letting the constrained generation loop produce them — this technically produces valid-looking output but isn't actually the model's own generated choice.

Done Criteria

- ☐ All 11 example test prompts produce the correct function name and correctly-extracted arguments.
- ☐ No code path in function or parameter selection relies on string matching, keyword detection, or regex against the original prompt — every choice traces back to masked model-generated tokens.
- ☐ A new, unseen prompt targeting an existing function in the schema is correctly handled, demonstrating the approach generalizes rather than memorizing the given examples.

What This Unlocks

Chapter 8 (The Full Generation Pipeline), which orchestrates this now-correct single-prompt behavior across a full batch of prompts efficiently.

Chapter 8 — The Full Generation Pipeline

You Are Here

Single-prompt generation is fully correct: valid JSON, schema-compliant, and model-driven in its choices. This chapter is about orchestration — running this correctly across every prompt in a batch, efficiently, without repeating expensive setup work per prompt.

Why This Module Exists

The subject requires processing all test prompts in under 5 minutes on standard hardware, and expects a single output file containing one result per input prompt. Naively re-loading the model or re-parsing the schema for every single prompt would be wasteful and could push runtime well past a reasonable bound. This chapter is about correctly structuring the per-prompt work you've already built into an efficient batch process.

Conceptual Understanding

What Should Happen Once vs. Per-Prompt

Loading `Small_LLM_Model` (Chapter 1) is expensive and must happen exactly once per program run, not once per prompt. Loading and parsing `functions_definition.json` (Chapter 6) should also happen once. Building the vocabulary token-to-string map (Chapter 3) should happen once. What must happen per-prompt is: constructing that specific prompt's generation text (Chapter 7), running the constrained generation loop (Chapters 2, 4, 5, 6) fresh for that prompt, and recording its result.

Why State Must Reset Between Prompts

The JSON grammar state machine (Chapter 4) and the schema-tracking state (Chapter 6) both need to start fresh for every new prompt — a new independent JSON object is being generated each time. Carrying state over from a previous prompt's generation into the next would immediately corrupt masking decisions, since the new generation isn't a continuation of the old one.

Structuring the Pipeline Function

A clean structure separates setup (run once) from the per-prompt loop (run once per prompt), and keeps a single, obvious function boundary around "generate one function call for one prompt," which Chapter 7 already built and tested in isolation.

```
def run_pipeline(functions_definition_path, prompts, model):
    schema = load_schema(functions_definition_path)      # once
    vocab_map = build_vocab_map(model)                  # once
    results = []
    for prompt_entry in prompts:                        # per-prompt
        fresh_json_state = JsonState()
        fresh_schema_state = SchemaState(schema)
        result = generate_one(prompt_entry['prompt'], schema,
                              model, vocab_map,
                              fresh_json_state, fresh_schema_state)
        results.append({'prompt': prompt_entry['prompt'], **result})
    return results
```

Isolating Failures Per-Prompt

If generation fails or produces an error for one prompt, that failure should not crash the entire batch and lose results already computed for earlier prompts. Each prompt's generation should be wrapped so that a failure is caught, logged or recorded in a way that satisfies the subject's error-handling

requirements (covered in full in Chapter 10), and the pipeline continues to the next prompt rather than aborting entirely.

Conceptual Checkpoint

1. List which parts of the pipeline must run exactly once per program execution, and which must run once per prompt.
2. Why must the JSON grammar state and schema state both be reset before generating a new prompt's output?
3. Why should a single prompt's generation failure not be allowed to crash the entire batch?

Implementation Goals

- ☐ A pipeline function that loads the model, schema, and vocabulary map exactly once, then iterates over all prompts.
- ☐ Fresh JSON grammar state and schema state instantiated at the start of every individual prompt's generation.
- ☐ Per-prompt error isolation, so one failing prompt doesn't prevent results from other prompts being recorded.
- ☐ A results structure accumulating {prompt, name, parameters} per prompt, ready to be handed to the output writer (Chapter 9).

System Reconnection — Where does this fit into the whole project?

This chapter is the orchestration layer sitting just outside the Constrained Generation Loop box in the architecture diagram — it's what calls that loop once per prompt, in the main request journey described in the Final System Integration walkthrough. It doesn't change how a single generation works; it changes how many times, and how cleanly, that single-generation logic gets invoked.

Testing & Verification

- Run the full pipeline against the complete `function_calling_tests.json` file (11 prompts) and confirm 11 results are produced, each independently correct.
- Time the full run and confirm it comfortably fits within the subject's 5-minute benchmark on your hardware.
- Deliberately inject a prompt likely to cause a generation issue (e.g. an empty string) partway through a batch and confirm the rest of the batch still completes and produces results.

Common Mistakes & Debugging Strategies

- Reloading `Small_LLM_Model` inside the per-prompt loop, causing massive unnecessary slowdown across a batch.
- Forgetting to reset JSON/schema state between prompts, causing later prompts in a batch to fail or behave strangely due to leftover state from earlier prompts.
- Letting one prompt's exception propagate uncaught, silently losing all results computed for prompts before it in the batch.

Done Criteria

- ☐ The full example test file (11 prompts) is processed correctly and efficiently in a single pipeline run.

- ☐ The model, schema, and vocabulary map are demonstrably loaded only once per run (e.g. by confirming via a print statement or profiler that setup code executes exactly once).
- ☐ A deliberately problematic prompt inserted into a batch does not prevent the rest of the batch from completing.

What This Unlocks

Chapter 9 (Input/Output Contract and CLI), which wraps this pipeline in the exact command-line interface and file I/O contract the subject requires.

Chapter 9 — Input/Output Contract and CLI

You Are Here

The pipeline correctly and efficiently processes a batch of prompts into correct results. This chapter wraps that pipeline in exactly the command-line interface and file format the subject specifies — matching argument names, default paths, and output structure precisely, since these are graded exactly, not approximately.

Why This Module Exists

The subject specifies an exact invocation command, exact default directories, and an exact output file shape with exact key names. A pipeline that works correctly internally but doesn't match this contract will fail automated review (the moulinette) even if the underlying logic is sound. This chapter is about precision to specification, not new algorithmic content.

Conceptual Understanding

The Exact Required Invocation

Per the subject, the program must run via: `uv run python -m src [--functions_definition <function_definition_file>] [--input <input_file>] [--output <output_file>]`. All three arguments are optional/flagged, meaning the program must work correctly when none are provided, using default paths.

Default Paths

By default, input files are read from `data/input/` and output is written to `data/output/`. This means without any flags, the program should look for `data/input/functions_definition.json` and process whatever input file convention you've established there, writing to `data/output/function_calling_results.json` specifically — that exact filename is specified by the subject.

Exact Output Shape

The output file must be a single JSON file containing an array, where each object has exactly three keys: `prompt` (string, the original request), `name` (string, the chosen function), and `parameters` (object, the extracted arguments with correct types). No extra keys, no missing keys, no prose anywhere in the file — the subject explicitly forbids extra keys or prose anywhere in the output.

```
[
  {
    "prompt": "What is the sum of 2 and 3?",
    "name": "fn_add_numbers",
    "parameters": {"a": 2, "b": 3}
  },
  ...
]
```

Argument Parsing Structure

Using Python's `argparse` (or an equivalent library), define the three optional arguments with the exact flag names the subject specifies, each defaulting to the documented default path when omitted. The module must be runnable via ``python -m src``, meaning your `src` directory needs the appropriate `__main__.py` or equivalent entry point wiring for that invocation style to work.

Conceptual Checkpoint

1. State the exact invocation command the subject requires, including all three optional flags.
2. What are the default directories used when `--input` and `--output` are not specified?
3. What exact filename must the output file have, and what are its three required top-level keys, in the exact required types?
4. What does ``python -m src`` require of your directory/module structure for that invocation to work at all?

Implementation Goals

- ☐ argparse-based (or equivalent) CLI parsing for `--functions_definition`, `--input`, and `--output`, each defaulting correctly when omitted.
- ☐ A `__main__.py` (or equivalent) making ``python -m src`` a valid invocation of your program.
- ☐ An output writer producing exactly the required JSON array shape, with exactly the three required keys per object and no others.
- ☐ Verification that running with zero flags correctly falls back to the documented default paths.

System Reconnection — Where does this fit into the whole project?

This chapter is the `main()` / CLI box at the very top of the architecture diagram, and the Output Writer box at the bottom. It's the outer shell around the entire pipeline built in Chapters 1-8 — everything else remains unchanged; this chapter makes sure the outside world can actually invoke and consume that pipeline correctly.

Testing & Verification

- Run the program with zero flags and confirm it correctly reads from `data/input/` and writes to `data/output/function_calling_results.json`.
- Run the program with all three flags explicitly pointing at custom file paths and confirm it correctly uses those instead of the defaults.
- Open the produced output file and confirm, key by key, that every object has exactly `prompt`, `name`, and `parameters`, with no extra fields and no stray text anywhere in the file.

Common Mistakes & Debugging Strategies

- Using different flag names than exactly what the subject specifies (`--functions_definition`, `--input`, `--output`) — even close variants will fail automated grading that invokes the program with the exact documented command.
- Writing extra debug fields (e.g. a confidence score, or timing info) into the output JSON objects — the subject explicitly forbids extra keys anywhere in the output.
- Hardcoding `data/input/` or `data/output/` paths in a way that ignores the `--input/--output` flags when they are actually provided.

Done Criteria

- ☐ Running ``uv run python -m src`` with no arguments correctly processes default input files and writes correctly to the default output path.

- ☐ Running with explicit `--functions_definition`, `--input`, and `--output` flags correctly overrides the defaults.
- ☐ The output file's JSON structure exactly matches the required shape: an array of objects with exactly `prompt`, `name`, and `parameters` as keys.

What This Unlocks

Chapter 10 (Robustness, Errors, and Packaging), which hardens this now-functionally-complete tool against malformed inputs and prepares it for submission.

Chapter 10 — Robustness, Errors, and Packaging

You Are Here

The tool works correctly end-to-end on well-formed inputs, with the exact required CLI and output contract. This final chapter hardens it against malformed or missing inputs, adds the required validation and linting compliance, and prepares the project structure for submission.

Why This Module Exists

The subject requires that the program never crash unexpectedly, always provide clear error messages, and gracefully handle malformed or missing input files. It also mandates specific tooling compliance (flake8, mypy with specific flags, pydantic validation, a specific Makefile structure) and a specific repository structure for submission. None of this changes the core algorithm — it's what makes the difference between a correct prototype and a submittable, gradeable project.

Conceptual Understanding

What 'Graceful' Error Handling Means Here

Per the subject: your functions should handle exceptions gracefully using try-except blocks, prefer context managers for resources like files, and never crash unexpectedly during review — an unhandled crash means the activity is considered non-functional. This applies specifically to: missing input files, malformed JSON in input files, and any generation-time failure for a specific prompt (which Chapter 8 already isolates per-prompt, but must still surface a clear message rather than silently failing).

Where pydantic Fits

The subject requires all classes to use pydantic for validation. This applies naturally to your schema representation (Chapter 6) and your output record representation (Chapter 9) — defining these as pydantic models means malformed data (wrong types, missing required fields) is caught automatically at construction time with a clear validation error, rather than causing confusing failures deeper in the pipeline.

Type Hints and mypy Compliance

Every function parameter, return type, and applicable variable needs type hints using the typing module, and all functions must pass mypy with the exact flags the subject specifies: `--warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defs`. This is not optional polish — code that fails this lint step fails the subject's Common Instructions regardless of runtime correctness.

The Required Makefile Targets

install (dependency installation via uv/pip/pipx), run (execute the main script), debug (run under Python's built-in debugger), clean (remove `__pycache__`, `.mypy_cache`, etc.), lint (flake8 and the exact mypy invocation above), and optionally lint-strict. Each target must actually work when invoked via `make <target>`.

Repository Structure for Submission

Per the subject: a `src/` directory with your implementation, `pyproject.toml` and `uv.lock` for dependency management, an `llm_sdk/` directory (copied from the provided package), a `data/input/` directory with

test files for demonstration, a README.md with comprehensive documentation (see the Readme Requirements chapter of the subject for the exact required sections), and explicitly no output/ directory committed — it must be generated fresh during review, not pre-committed.

Conceptual Checkpoint

1. Name at least three distinct failure scenarios your error handling must cover, per the subject's requirements.
2. Where does pydantic validation naturally fit in this project's architecture, and what does it protect you against?
3. What are the exact mypy flags the subject requires the lint target to run with?
4. What must NOT be included in your submitted repository, and why?

Implementation Goals

- ☐ try/except coverage (with context managers for file handling) around all file I/O: loading functions_definition.json, loading the input prompts file, and writing the output file.
- ☐ pydantic models for the function schema and the output record shape, replacing any plain dictionaries used for these during earlier development.
- ☐ Full type hints across all functions, verified against mypy with the subject's exact required flags.
- ☐ flake8 compliance across the codebase.
- ☐ A Makefile with install, run, debug, clean, lint (and optionally lint-strict) targets, each verified to actually work.
- ☐ A .gitignore excluding Python artifacts (__pycache__, .mypy_cache, etc.).
- ☐ Final repository structure matching the subject's required layout, with output/ explicitly excluded.
- ☐ A complete README.md covering all sections required by both the Common Instructions and this project's specific additional requirements (algorithm explanation, design decisions, performance analysis, challenges faced, testing strategy, example usage).

System Reconnection — Where does this fit into the whole project?

This chapter doesn't add a new functional box to the architecture diagram — it wraps every existing box (CLI, pipeline, generation loop, output writer) in the error handling and validation that make the whole system trustworthy under real-world, imperfect input, and packages the whole thing for submission and peer review.

Testing & Verification

- Run the program pointing --functions_definition at a nonexistent file path and confirm a clear error message is shown, with no unhandled crash/traceback exposed to the user.
- Run the program against a deliberately malformed JSON input file (e.g. a trailing comma) and confirm the same graceful handling.
- Run the program against an empty prompts array and confirm it completes cleanly, producing an empty (but valid) results file rather than erroring.
- Run make lint and make lint-strict and confirm both complete without errors.

- From a completely fresh clone/copy of your repository, run only uv sync followed by the documented invocation command, and confirm the program runs correctly with no additional manual setup steps.

Common Mistakes & Debugging Strategies

- Catching exceptions too broadly (a bare except:) and silently swallowing real bugs instead of surfacing a clear, specific error message.
- Leaving plain dictionaries in place for schema or output data instead of migrating to pydantic models, missing the subject's explicit requirement.
- Forgetting to test the project from a truly fresh environment (only uv sync, nothing else) before submission — a project that only works because of leftover local state or manually-installed packages will fail review.
- Accidentally committing the output/ directory, which the subject explicitly says must not be included.

✓ Done Criteria

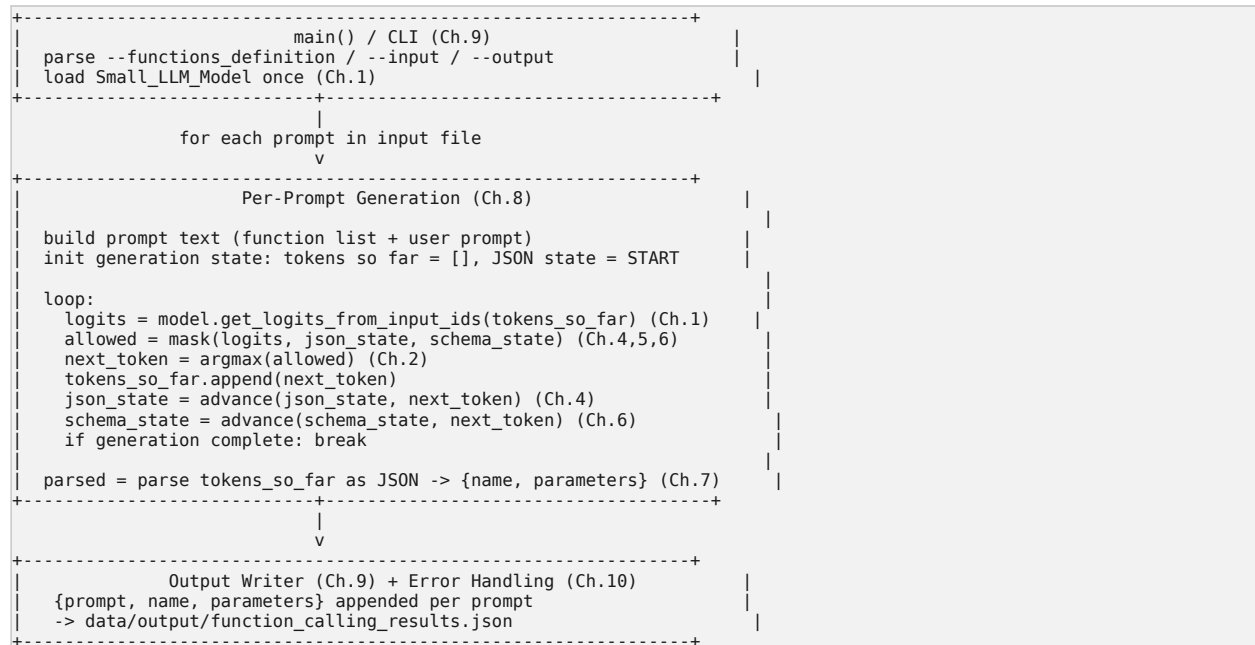
- ☐ The program never crashes with an unhandled exception under any tested malformed-input scenario — every failure produces a clear, specific message instead.
- ☐ make lint and make lint-strict both pass with zero errors.
- ☐ A fresh clone of the repository, set up with only uv sync, runs correctly using the documented default invocation.
- ☐ The repository structure exactly matches the subject's required layout, with output/ excluded.
- ☐ README.md contains every required section from both the Common Instructions and this project's specific additional requirements.

What This Unlocks

Full System Integration — every module now exists and is individually correct; the remaining work is confirming the whole pipeline behaves correctly together, end to end, exactly as a reviewer or the moulinette would run it.

Final System Integration

Fully Annotated Architecture Diagram



Full Request Journey (End-to-End Walkthrough)

1. `main()` parses CLI args (`--functions_definition`, `--input`, `--output`), applying defaults of `data/input/` and `data/output/` if omitted (Ch.9).
2. The `functions_definition.json` file is loaded and validated — every function's name, parameter names/types, and description are read into an in-memory schema representation (Ch.6).
3. The input prompts file is loaded; each prompt is validated as a non-empty string (Ch.9, Ch.10).
4. `Small_LLM_Model` is instantiated once (not once per prompt) via `llm_sdk` (Ch.1).
5. For each prompt: a full generation prompt string is constructed, combining the function definitions and the user's natural-language request, then encoded to input token IDs (Ch.1).
6. The manual generation loop begins: at every step, `get_logits_from_input_ids()` is called on the tokens generated so far (Ch.1, Ch.2).
7. Before any token is chosen, the JSON grammar state machine and the schema state machine are both consulted to determine which tokens are currently allowed (Ch.4, Ch.6).
8. Every disallowed token's logit is set to negative infinity; the token with the highest remaining logit is selected (Ch.5).
9. The chosen token is appended to the sequence, both state machines advance, and the loop repeats until a complete, valid function call has been generated (Ch.2, Ch.4, Ch.6).
10. The completed token sequence is decoded and parsed into `{name, parameters}`, using the LLM's own choice — never a heuristic or string match (Ch.7).
11. The result, combined with the original prompt, is appended to the output structure; after all prompts are processed, the output is written as a single valid JSON file (Ch.9).

12. Errors at any stage (malformed input files, missing files, generation failures) are caught and reported clearly without crashing the program (Ch.10).

Final Integration Checklist

- ☐ The program never calls a high-level `.generate()` convenience method — generation is manual, token-by-token, using only `get_logits_from_input_ids()`.
- ☐ Every generated output is valid, parseable JSON — 100% of the time, not just most of the time.
- ☐ Every generated output matches the `functions_definition.json` schema exactly: valid function name, all required parameters present, correct types.
- ☐ The function and parameters are chosen by the LLM's own token generation — no heuristic string-matching or keyword detection is used to pick the function.
- ☐ The vocabulary file is used to determine which token IDs correspond to which strings, and that mapping drives the masking logic.
- ☐ All classes use `pydantic` for validation, per the subject's additional requirements.
- ☐ Only `numpy`, `json`, and `pydantic` are used beyond `llm_sdk` itself — no `torch`, `transformers`, `huggingface`, `dspy`, or `outlines` imported directly in your own code.
- ☐ No private methods or attributes of `llm_sdk` are accessed (nothing prefixed with a single underscore).
- ☐ `flake8` and `mypy` (per the required lint flags) both pass with zero errors.
- ☐ The program never crashes on malformed or missing input files — it reports a clear error instead.
- ☐ `uv sync` alone is sufficient to set up the environment, and the `Makefile`'s `install/run/debug/clean/lint` targets all work.
- ☐ `README.md` includes all required sections, including the project-specific ones: algorithm explanation, design decisions, performance analysis, challenges faced, testing strategy, and example usage.

Appendix

Glossary (In Order Introduced)

Token — a subword unit produced by the tokenizer; the smallest unit the model reads or generates, not necessarily a whole word.

Logits — raw, unnormalized scores the model produces for every possible next token, before any conversion to probabilities.

Autoregressive generation — generating output one token at a time, where each new token is chosen based on all tokens generated so far, and is then fed back in as part of the input for the next step.

Argmax — the operation that returns the index (here, the token) with the highest score in a list.

Vocabulary file — a file mapping every token ID the model can produce to its string representation.

Finite state machine (as used here) — a model of JSON-parsing progress that tracks, at any point during generation, exactly what kinds of characters/tokens are syntactically valid next, given everything generated so far.

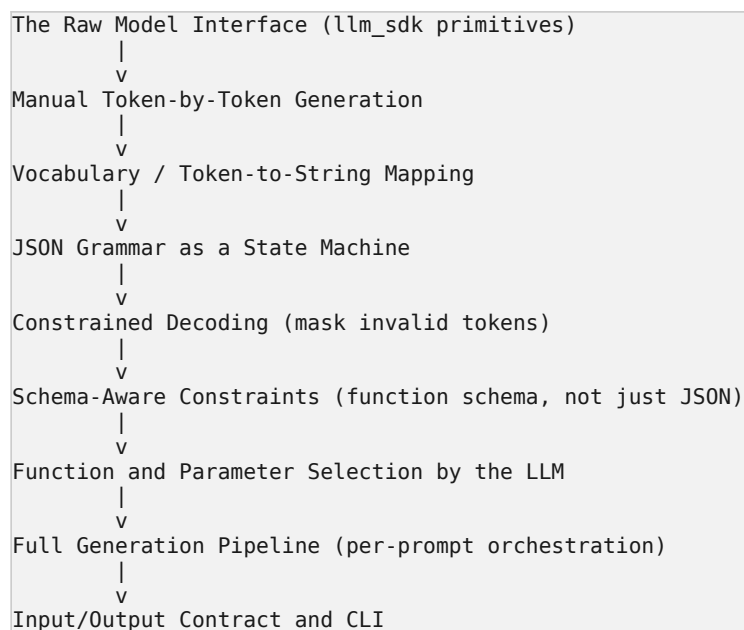
Constrained decoding — modifying a model's logits before token selection so that only tokens consistent with a required structure (here, JSON syntax and a function schema) can ever be chosen.

Masking — the act of setting the logits of disallowed tokens to negative infinity, so they can never be selected by argmax or any probability-based sampling.

Schema — a description of the exact structure a valid function call must have: which function names exist, which parameters each requires, and what type each parameter must be.

Prefix-validity — the property that a partial (incomplete) generated sequence is still consistent with eventually becoming valid JSON — the property the state machine must track at every step, not just at the end.

Full Dependency Graph (Final)





Study & Implementation Timetable

Assumptions: This schedule assumes 4 focused hours per day. If you have fewer hours available per day, extend the total number of days proportionally rather than skipping or compressing chapters — the constrained-decoding core (Days 4-7) is the highest-risk, highest-value part of this project and should not be rushed.

Total: 10 days, organized into 4 phases.

Phase 1 — Foundations: Talking to the Model (Days 1-2)

Day	Focus	Must Understand	Must Implement	End-of-Day Test
1	Ch.1: The Raw Model Interface	What the SDK actually provides (encode, decode, get_logits_from_input_ids, vocab file path) and what it deliberately does NOT provide (no generate())	A small script that encodes a prompt, calls get_logits_from_input_ids once, and prints the top 5 highest-logit tokens	Running the script on a simple prompt produces 5 plausible-looking next tokens when manually inspected
2	Ch.2: Manual Token-by-Token Generation	Autoregressive generation as a loop; argmax token selection; why generated tokens must be fed back into the next call	A generation loop that repeatedly calls get_logits_from_input_ids, argmaxes, appends, and stops at a max length or EOS token, with no constraints yet	Running the loop on an open-ended prompt produces coherent (not necessarily JSON) text of the expected length

Phase 2 — Mapping the Model's Language (Day 3)

Day	Focus	Must Understand	Must Implement	End-of-Day Test
3	Ch.3: Vocabulary and Token-to-String Mapping	How to load and use the vocabulary file to go from token ID to string and back; why this mapping must be fast (dict/set-based, not linear search)	A function mapping every token ID in the vocabulary to its string form, and a reverse lookup; a helper answering 'does token ID X decode to a character that would violate JSON syntax right now'	For a handful of hand-picked token IDs, the printed string form matches what you'd expect by manual inspection of the vocab file

Phase 3 — Constraining the Output (Days 4-7)

Day	Focus	Must Understand	Must Implement	End-of-Day Test
4	Ch.4: JSON Grammar as a State Machine	The full set of JSON states (start, inside object, expecting key, expecting colon, expecting value, inside string, inside number, after value, expecting comma-or-close, etc.) and the legal transitions between them	A JSON state-tracking object with a single 'advance(character)' method and a method reporting which characters are currently valid next	Manually feed the state machine a few valid and invalid JSON strings character-by-character; it correctly flags exactly where the invalid ones break
5	Ch.5: Constrained Decoding — Masking Invalid Tokens	Why masking happens on logits (before selection), not by filtering after generation; why negative infinity is used specifically	A masking function that, given current logits, the JSON state, and the vocabulary map, sets every logit for a currently-invalid token to negative infinity before argmax runs	Running the full constrained loop (Ch.2 + Ch.4 + Ch.5 together) on a prompt asking for JSON produces syntactically valid JSON, every single run, across at least 20 repeated attempts
6	Ch.6: Schema-Aware Constraints	How a function schema (name, parameter names, parameter types) narrows the JSON grammar further — e.g. only certain keys are valid inside "parameters", only certain value types are valid per key	Schema-tracking logic layered on top of the JSON state machine, restricting valid tokens further based on the loaded functions_definition.json	Constrained generation against a real functions_definition.json produces output where every key and value type exactly matches the schema, across multiple different functions
7	Ch.7: Function and Parameter Selection by the LLM	Why the function name itself must be chosen by the model's own generated tokens, not matched via string search against the prompt	The full single-prompt pipeline: prompt in, valid schema-compliant {name, parameters} out, with the LLM making every choice	For each of the 11 example prompts in function_calling_tests.json,

Day	Focus	Must Understand	Must Implement	End-of-Day Test
				the correct function and correctly-typed arguments are produced without any hardcoded prompt-to-function mapping

Phase 4 — Assembling the Full Tool (Days 8-10)

Day	Focus	Must Understand	Must Implement	End-of-Day Test
8	Ch.8: The Full Generation Pipeline	How per-prompt generation (Ch.7) is orchestrated across a whole batch of prompts efficiently (e.g. loading the model once, not once per prompt)	A pipeline function that takes a list of prompts and a schema, and returns a list of {prompt, name, parameters} results	Running the pipeline against the full function_calling_tests.json file produces one correct result per prompt, with the model loaded only once
9	Ch.9: Input/Output Contract and CLI	The exact required CLI invocation shape and default paths; the exact required output JSON shape and key names	argparse-based CLI matching the subject's exact invocation format; a writer producing data/output/function_calling_results.json in the exact required shape	Running <code>uv run python -m src` with no arguments correctly reads from data/input/ and writes a correctly-shaped file to data/output/</code>
10	Ch.10 + Final Integration: Robustness, Errors, Packaging *(buffer-heavy — treat as high-risk)*	What 'graceful' error handling means here: never crashing, always a clear message, for missing files, malformed JSON, and generation edge cases	try/except coverage around all file I/O and generation calls; pydantic validation on schema and output models; flake8/mypy passing; Makefile targets; README with all required sections	Deliberately pass a missing file, a malformed JSON file, and an empty prompts list — the program reports a clear error

Day	Focus	Must Understand	Must Implement	End-of-Day Test
				each time and does not crash. Full Final Integration Checklist passes top to bottom

Why Days 4-7 Are Not Compressed

Days 1-3 build the raw plumbing to talk to the model. Days 4-7 are the actual intellectual core of this project — turning an unreliable small model into a 100%-reliable structured-output generator through grammar and schema constraints. This is the skill the subject explicitly says it wants you to develop, and it is also the part most likely to contain subtle bugs (a state machine transition missed, a token boundary miscounted). Budget the full four days; do not attempt to collapse JSON-grammar masking and schema masking into the same day even if both feel conceptually similar — building and testing them separately (Ch.5 before Ch.6) makes debugging dramatically easier, because a schema violation and a raw JSON syntax violation look identical in the output unless you already know Ch.5's layer is independently correct.

If You Have Fewer Days Than Planned

Do not remove chapters. If you're short on time, merge Days 1-2 into a single day if you're already comfortable with basic model inference concepts, and merge Day 3 into Day 4 if you're comfortable with dictionary-based lookups already. Do not compress Days 4-7 under any circumstances — this is where correctness actually lives, and rushing it produces a tool that appears to work on the example prompts but fails unpredictably on the prompts used during peer review, which the subject explicitly warns will differ from the examples given.